Q26: Fine-Tuning and Adaptation

Paradise (nuc13)

October 2026

The questions

- How does fine-tuning change a model's behavior?
- Which post-training pipelines adapt well to new domains?
- How do we tune systems online?

Answer preview: **tuning moves the response distribution more than the knowledge; keep the update bounded; retrieve for facts, tune for behavior; SFT before preference.**

Weight update vs. behavior surface

- Mechanics: SGD on a new loss from pretrained θ_0 — says nothing about *what* changes
- **Behavior reshaping:** instruction tuning / preference opt move the response distribution
  - InstructGPT 1.3B *preferred* over GPT-3 175B (Hu et al. 2022 is LoRA; Ouyang et al. 2022)
  - DPO: classification-like loss, KL anchor to reference
- **Knowledge injection:** much harder — unsupervised FT learns new facts poorly

Catastrophic forgetting

- Sequential training on B overwrites weights that A relied on — abruptly, not gradually (French 1999)
- Plain SGD has no notion of "important to old task" (Kirkpatrick et al. 2017, EWC)
- Mitigations that matter in LLM practice:
  1. Replay: mix old/general data into domain training
  2. Constrained updates: EWC penalty · importance; LoRA by construction

LoRA: low-rank adapters

$$W = W_0 + \frac{\alpha}{r} BA, \quad B \in \mathbb{R}^{d \times r},\ A \in \mathbb{R}^{r \times k},\ r \ll \min(d, k)$$

- B init at zero $\Rightarrow \Delta W = 0$ at start; only A, B get gradients
- Up to 10,000$\times$ fewer trainable params on GPT-3; **zero added inference latency** (merge $W_0 + BA$)
- Hu et al. 2022: adaptation ΔW has low "intrinsic rank" — tiny r suffices
- **Why capabilities survive:** residual confined to an r-dim subspace; base directions untouched $\Rightarrow$ soft EWC

QLoRA and the PEFT family

- QLoRA (Dettmers et al. 2023): backprop through a frozen **4-bit** quantized base into LoRA
- NF4 datatype + double quantization + paged optimizers
- 65B model fine-tuned on one 48GB GPU, 16-bit-level task performance
- Family view: adapters (Houlsby), prefix/prompt tuning, LoRA, QLoRA — all share "freeze the base, learn a small edit"
- Bonus: adapters are swappable per domain; one base serves all

RAG vs. fine-tune

The rule

If it's facts you need, retrieve; if it's behavior you need, tune.

- Ovadia et al. 2024: RAG beats unsupervised FT for knowledge injection — seen and unseen knowledge alike
- Retrieve: volatile, citable, large, revocable knowledge (Lewis et al. 2020)
- Tune: format, style, domain instruction-following, latency-critical skills
- Combine when both apply: LoRA generator over a domain index

Online / continual adaptation

- **Test-time training** (Sun et al. 2020): each test input becomes a self-supervised problem; model updated before predicting; extends to streams
- **Streaming weight updates:** fragile — drift, small batches, forgetting at production cadence
- **Why naive online SGD breaks RLHF models:**
  1. No KL anchor $\Rightarrow$ policy drifts past what preference data certified
  2. Likelihood updates on a narrow stream sharpen/collapse entropy
  3. Compounding: each update moves the next one's starting point
- Deployed systems use bounded edits: small-r LoRA, anchors, periodic re-tune from frozen base

Pipeline order in practice

1. **Retrieval first** — cheapest iteration loop; measure the frozen base + RAG gap
2. **SFT for domain behavior** — LoRA/QLoRA, replay data mixed in
3. **Preference optimization last** — DPO/RLHF assumes SFT-level competence; anchor defined against post-SFT reference

Reversing 2 and 3 wastes the preference stage re-teaching format.

Reading

- Hu et al. 2022 (LoRA); Dettmers et al. 2023 (QLoRA)
- Kirkpatrick et al. 2017 (EWC / catastrophic forgetting); French 1999 (survey)
- Lewis et al. 2020 (RAG); Ovadia et al. 2024 (fine-tune vs. retrieval)
- Sun et al. 2020 (test-time training); Ouyang et al. 2022; Rafailov et al. 2023